

Supply dispatch audit

Practical Lab 1 · C++ foundations · Independent assessment

A game-support team needs a console tool to audit supplies before dispatching them. A depot can request more units than it has in stock, input can be mistyped, and a batch file can be incomplete. Your program must report what can actually be dispatched without confusing an invalid file with a valid small stockholding.

Build the program from this specification. A separate, deliberately faulty project tests whether you can diagnose build failures. There is no worked implementation to follow.

Build and diagnose — separate files, declarations and definitions.

Read and validate — console responses and a bounded batch file.

Update and explain — scalar values, pointer targets and safe access.

Your working agreement

Allow approximately **two hours** for the whole assessment. Complete Tasks A–E; all are included in the **100 marks**. Basic, intermediate and expert describe the quality of your evidence, not different worksheets or extra topics. A partial implementation can earn credit for requirements it genuinely meets.

Use **Windows, Visual Studio, C++17, Debug, x64 and IW4**, with GitHub access already available. The supplied projects target the v143 toolset; retarget to an installed compatible toolset when necessary. Opening a project and resolving an installation problem are not additional programming challenges.

Work independently under the collaboration, documentation and AI conditions published with the Blackboard assessment. Write your own implementation rather than transplanting a completed tutorial. Supplied data and project configuration may be reused. Appropriate assistive input remains permitted under your approved arrangements.

What you are demonstrating

You will **implement** a stream-based program, **diagnose** compilation and linking failures, **trace** changes through a non-owning pointer, and **justify** tests against a stated contract. Here, a **contract** means the input rules and required behaviour against which your program is assessed.

No arrays, reference parameters, enumerations, custom classes, dynamic allocation, file output or game engine are required. Use the scalar variables, conditions, loops and functions already available to you.

A · Build a maintainable program and diagnose faults

15 marks · Required artefacts: two repaired/implemented projects and a short diagnostic record

The `Lab_01` folder contains two separate solutions. Keep them separate; do not add both `Main.cpp` files to one executable project. The source files in `DispatchAudit` are intentionally unimplemented. The `BuildInvestigation` files contain a small diagnostic specimen, not a solution to the audit.

A1 · Your audit project

Use the following organisation. The heading wording is your choice, but the function must be called by the executable.

File	Required responsibility
<code>DispatchAudit/Main.cpp</code>	One main function; console input, file processing and the dispatch demonstration
<code>DispatchAudit/Dispatch.h</code>	Guarded declaration of <code>void printHeading();</code> inside namespace <code>Dispatch</code>
<code>DispatchAudit/Dispatch.cpp</code>	The matching definition of <code>Dispatch::printHeading</code>
<code>DispatchAudit/batches.txt</code>	The active input data, beside the project file
<code>evidence.md</code>	Your diagnostic record, test results and explanations

Use standard-library names with `std::`. Do not place a `using namespace std;` directive in the header. Initialise variables before use and use consistent four-space indentation. Comments should explain decisions or rules, not narrate every assignment.

A2 · Investigate the supplied build

The intended `BuildInvestigation` executable prints its diagnostic heading followed by `Packages: 3`. It contains **two independent faults**. Diagnose and repair both without changing that intended behaviour or removing the call to `Banner::show`.

For each fault, record the first useful diagnostic, the stage that detected it, the file and cause, and the change that repaired it. Exact diagnostic codes may differ between compiler versions. Include a final successful build and output record; do not run an old executable after a failed build.

A3 · Explain the build you produced

In no more than **80 words**, identify the language standard, compiler and development environment you used. Explain how the header declaration and source-file definition let `Main.cpp` call the function, and distinguish an ordinary incremental Build from Rebuild. Use your own file names in the explanation.

Evidence boundary: a successful build shows that compilation and linking completed. Your later tests must establish whether the program meets the data and dispatch requirements.

B · Accept a depot name and a dispatch request

20 marks · Required behaviour: retain the name, recover from invalid responses and stop on unavailable input

Read the depot name first, then the number of units requested. Keep prompts understandable. Ordinary information belongs on standard output; terminal input failures belong on standard error. Do not build a menu or a repeated dispatch system: one accepted request per run is sufficient.

Input	Contract
Depot name	Read a complete line into a <code>std::string</code> , preserving spaces. A successfully read empty line is permitted. If no line can be read, report the failure and return a non-zero exit status.
Requested units	An <code>int</code> from 0 to 60 inclusive . Retry after an unreadable or out-of-range response.
Recovery	A retry must make progress past the failed response. Do not repeatedly attempt the same bad characters.
Ended/unavailable input	Stop with an error rather than loop indefinitely.
Accepted numeric response	Use the numeric-prefix policy below and discard the remainder of that response line.

Numeric-prefix policy

An accepted integer prefix is the request. For example, `25abc` means **25**, and the remaining `abc` is discarded. `abc25` has no readable integer prefix and requires another response. Likewise, `80abc` is outside the permitted range and requires a retry. A response such as `25 99` supplies one request, **25**, not two dispatches.

This is deliberately not strict whole-line number validation. Do not add a new parser that rejects an explicitly accepted response.

Interactions your program must support

Fresh responses, one per line	Required result
North Depot, then 25	Preserve the full name and accept 25.
North Depot, then abc, 80, 25	Two retries, then accept 25.
North Depot, then 25abc	Accept 25 and discard the rest of that line.
A name, then input closes before a valid request	Report the ended input; no completed dispatch report.

For input that closes immediately after an accepted valid integer, the request is still usable. You do not need any later console input. Write a two-sentence explanation of why resetting a stream's state and discarding input are different actions.

C · Audit the batch file before reporting success

25 marks · Required behaviour: a complete validated stock total, not a successful partial report

Read `batches.txt` relative to the program's working directory. The supplied project sets that directory to its project folder when launched through Visual Studio. The physical location of the file matters; its appearance in Solution Explorer is not sufficient evidence that it can be opened.

File rule	Required interpretation
Format	A sequence of whitespace-separated integer entries; spaces, tabs and blank lines are permitted. Use the taught formatted-integer input model, not a new line-oriented parser.
Number of entries	1–12 inclusive . A thirteenth entry invalidates the audit.
Value of each entry	0–40 inclusive . Each entry represents units in one batch.
Low batch	An entry strictly below 10 , including zero.
Normal end	A final valid number may have a trailing newline or no trailing newline. Both are accepted.
Invalid data	A failed numeric extraction, an unrepresentable integer, an out-of-range entry or an excess entry cancels the audit.
No entries	An empty or whitespace-only file is a separate error, not a zero-unit valid audit.

Calculate the **batch count**, **original stock total** and **low-batch count**. Preserve the original stock total for the final report. The stated limits keep this total at most 480; no larger-number algorithm is needed.

Errors must remain distinguishable

Distinguish an open failure, a file with no entries, and invalid batch data. Also stop if the input stream reports a serious read failure. Error wording is flexible, but the cause must be recognisable. Return a non-zero status and do not print a completed stock/dispatch summary after any of these failures. A heading or prompt printed earlier is not a completed summary.

The file does **not** use the console's discard-the-rest-of-the-line policy. For example, `12abc 8` must not become a valid two-batch file by silently discarding `abc`. Do not treat a failed final number such as `+` or a very large integer as a clean end merely because the stream has reached the end of the file.

Baseline data

The supplied `batches.txt` contains **12, 0, 8 and 20**, on separate lines. A valid audit reports **4 batches, 40 units** and **2 low batches**. Other data sets are supplied in `data_cases`; they are inputs to test, not replacement implementations.

Acceptance rule: read and validate the whole file before applying a dispatch. An invalid later entry must not leave a successful-looking partial result.

D · Dispatch through a selected quantity

25 marks · Required behaviour: correct business result and a traceable pointer-based update

Dispatch the smaller of the accepted request and original stock. A shortage dispatches the available stock; it is not a file error. A zero request dispatches nothing. Count non-empty dispatches, not units: this single-request program produces a count of zero or one.

Required state and access constraints

Retain `totalStock` unchanged. Use a separate integer `availableStock`, initially equal to that total, and an integer `dispatchCount`, initially zero. Use **one non-owning `int*` named `selectedQuantity`** for the two updates below. A non-owning pointer accesses an existing object; it does not allocate or release it.

Operation	Requirement
Stock update	Select <code>availableStock</code> and subtract the dispatched amount through the pointer .
Dispatch counter	Retarget that same pointer to <code>dispatchCount</code> . Increment the pointed-to value once only when the dispatched amount is positive.
End of selection	Set the pointer to <code>nullptr</code> . A guarded output path must report that nothing is selected without dereferencing it.
Safety	Do not use <code>new</code> , <code>delete</code> , casts or arithmetic to jump between separate scalar variables. Do not execute an invalid pointer access.

Final report

Display the full depot name, batch count, low-batch count, original stock, requested units, dispatched units, remaining stock and dispatch count. Include a clear no-selection message after the final pointer reset. Labels and spacing may vary; numerical meaning must not.

Data and request	Dispatched	Remaining	Dispatch count
Baseline stock 40; request 25	25	15	1
Baseline stock 40; request 0	0	40	0
Stock 10 from 4 6; request 25	10	0	1
Stock 0 from 0 0; request 25	0	0	0

Trace your own execution

For the baseline run with request 25, record four states: before the stock update, after it, after retargeting and the possible increment, and after the null reset. At each state, identify the selected object, the relevant integer values, the pointer's stored value and its own address. Do not evaluate `*selectedQuantity` in the null state; record **not evaluated**.

Use Watch observations, a written table or an equivalent accessible trace. Explain why `selectedQuantity`, `*selectedQuantity` and `&selectedQuantity` are different expressions. Exact address numbers are not assessed. State why these objects do not require `delete`, and why a non-null value alone is not a universal safety guarantee.

E · Demonstrate, explain and submit

15 marks · Required artefacts: a compact test record, explanations and reproducible source

Run the seven named checks below and **one additional test you design**, giving eight recorded tests. Predict each outcome before running it, then record what actually happened. A failed test is useful evidence when you explain the defect honestly; do not relabel an unrun test as passed.

ID	Data / console input	Acceptance condition
P1	Baseline; North Depot, request 25	Full name; 4 batches, low count 2, original 40, dispatched 25, remaining 15, dispatch count 1.
P2	short-stock.txt; request 25	2 batches, low count 2, original 10, dispatched 10, remaining 0, count 1.
P3	Baseline; request 0	Original and remaining 40; dispatched 0, count 0.
P4	Baseline; requests abc, 80, 25abc	Recover twice, then accept 25 and produce the P1 quantities.
P5	batches.txt missing	Open error, non-zero exit, no completed report.
P6	empty.txt	No-entries error, non-zero exit, no completed report.
P7	malformed-final.txt	Invalid-data error, non-zero exit, no completed report.
P8	Your own case	Specify the input, the contract rule it tests and the expected result before running.

Additional checks may use whitespace-only input, an oversized entry, an out-of-range value, the maximum count, an excess entry, no final newline, a zero-stock file or ended console input. These are already published requirements, not undisclosed extensions. You are not required to cause a real hardware failure or perform an unsafe memory operation.

Evidence to include

Use `evidence.md` or export the HTML notes into an equivalent document. Include the two diagnostic records and the build explanation from A; the input-recovery explanation from B; the pointer trace and safety explanation from D; and the eight test records above. For one test, explain why your chosen input detects a mistake that the baseline would miss. Brief tables are sufficient; a long report is not required.

Submission package

Restore the baseline `batches.txt`, confirm which projects build, and commit your final work. Push to the authorised private repository. Submit the **repository URL and final commit identifier**, your evidence file, and a source ZIP through the assessment's Blackboard submission point. The ZIP provides a reproducible snapshot, not an alternative to explaining your code.

Keep `.cpp`, `.h`, `.sln`, `.vcxproj`, `.gitignore`, the data file and evidence. Exclude generated binaries, `bin`, `obj`, `.vs`, per-user files and credentials. Record any unfinished requirement and its location. HTML ticks and notes do not submit code or award marks; export notes before closing a shared computer session.

How the work is assessed

Mark allocation

Task	Marks	Evidence considered
A · Structure and diagnosis	15	Project/interface structure 5; two diagnosed repairs 6; build explanation 4
B · Console input	20	Full-line name 4; range and recovery 8; ended input 4; numeric-prefix policy 4
C · Batch audit	25	Valid aggregates 10; invalid/range/count handling 10; distinct missing/empty/read-failure handling 5
D · Dispatch and pointers	25	Dispatch quantities 8; pointer-based updates 9; trace, reset and safety explanation 8
E · Evidence and submission	15	Eight test records 7; targeted reasoning/readability 4; reproducible submission and Git evidence 4
Total	100	Partial credit is awarded within each criterion.

What different levels of understanding look like

Level	Typical evidence
Basic	A clear implementation of the normal case; genuine pointer use; some correct explanation. Recovery or boundary handling may be incomplete.
Intermediate	Normal and common failure cases handled consistently; accurate build diagnosis; a trace that separates the pointer from its target.
Expert	The full published contract is respected, including less obvious endings and zero cases; no unsafe access; tests and explanations identify why plausible alternatives fail.

These descriptors are not automatic percentage bands. A label, many commits or an attractive interface does not replace evidence. No extra library, optional tutorial extension or new language feature is needed for full marks.

Official documentation

Use these as reference material under the assessment's published conditions. They are not a prescribed solution sequence.

[Microsoft — translation units and linkage](#) · [input-stream operations](#) · [input file streams](#) · [raw pointers](#).